EXPERIMENT NO.1

AIM:

Program to input marks of 5 subjects and print the total and percentage using arrays

INTRODUCTION:

An array is a linear data structure that stores homogeneous data in contiguous memory locations. In this experiment the user will input marks in 5 subjects which will be stored in an array of size 5. The data in the array will then be added to find the total marks and percentage will be calculated out of 500.

SOURCE CODE:

```
#include<iostream.h>
#include<conio.h>
void main()
{
      float marks[5],total=0,per;
      clrscr();
      //Asking for marks from user and adding it to total
      for(int i=0;i<=4;i++)
      {
            cout<<"Enter marks in subject "<<i+1<<":";
            cin>>marks[i];
            total=total+marks[i];
      }
      //Calculation of percentage and printing
      per=total/5;
      cout<<"\nTotal="<<total<<" out of 500\nPercentage="<<per<<"%";
      getch();
}
```

SAMPLE OUTPUT:

Enter marks in Subject 1:90

Enter marks in Subject 2:90

Enter marks in Subject 3:90

Enter marks in Subject 4:90

Enter marks in Subject 5:90

Total=450 out of 500

EXPERIMENT NO.2

AIM:

Program to read two 3x3 matrices and add them

INTRODUCTION:

The user will be required to input the elements in the 3x3 matrices A and B. The elements of the matrices would be added and stored in matrix C. Nested for loops would be used for entering elements into the matrices and to calculate the sum matrix.

SOURCE CODE:

```
#include<iostream.h>
#include<conio.h>
void main()
{
	int a[3][3],b[3][3],c[3][3],i,j;
	clrscr();
	cout<<"MATRIX A\n";
	//Matrix values to be entered by user
	for(i=0;i<=2;i++)
	{		for(j=0;j<=2;j++)
		{
			cout<<"\nEnter element:";
			cin>>a[i][j];
		}
	}
	cout<<"\nMATRIX B\n";
	for(i=0;i<=2;i++)
	{	for(j=0;j<=2;j++)
		{
			cout<<"\nEnter element:";
			cin>>b[i][j];
		}
	}
	//Addition of two matrices
	cout<<"\nMATRIX C=MATRIX A+MATRIX B\n";
	for(i=0;i<=2;i++)
	{	cout<<"\n\n";
		for(int j=0;j<=2;j++)
		{	c[i][j]=a[i][j]+b[i][j];
			cout<<c[i][j]<<"\t";
		}
	}
	getch();
}
```

SAMPLE OUTPUT:

MATRIX A:

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

MATRIX B:

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

Enter element:2

MATRIX C=MATRIX A+MATRIX B

4	**4**	**4**
4	**4**	**4**
4	**4**	**4**

EXPERIMENT NO.3

AIM:

Program to find whether a matrix is upper triangular or not

INTRODUCTION:

In an nxn matrix, if all elements below the diagonal are 0, then it is called an upper triangular matrix. Similarly, in an nxn matrix, if all elements above the diagonal are 0, then it is called a lower triangular matrix.

Example:

$$\begin{pmatrix} 2 & 2 & 2 \\ 0 & 3 & 3 \\ 0 & 0 & 5 \end{pmatrix} \qquad \begin{pmatrix} 3 & 0 & 0 \\ 5 & 3 & 0 \\ 2 & 4 & 5 \end{pmatrix}$$

Upper Triangular Matrix Lower Triangular Matrix

SOURCE CODE:

```
#include<iostream.h>
#include<conio.h>
void main()
{
      int a[10][10],i,j,n,flag;
      clrscr();
      cout<<"Enter order of the matrix:";
      cin>>n;
      //Matrix values to be entered by the user
      cout<<"\nEnter matrix elements:\n";
      for(i=0;i<=n-1;i++)
      {
            for(j=0;j<=n-1;j++)
            {
                  cout<<"a["<<i<<"]["<<j<<"]=";
                  cin>>a[i][j];
            }
      }
      //Check for upper triangular matrix
      flag=1;
      for(i=0;i<=n-2;i++)
      {
            for(j=i+1;j<n;j++)
            {
                  if(a[j][i]!=0)
```

```
                {
                        flag=0;
                        break;
                }
            }
        }
        if(flag)
                cout<<"Upper Triangular Matrix";
        else
                cout<<"Not an Upper Triangular Matrix";
        getch();
}
```

SAMPLE OUTPUT 1:

Enter order of the matrix:3

Enter matrix elements:

a[0][0]=2

a[0][1]=2

a[0][2]=2

a[1][0]=2

a[1][1]=2

a[1][2]=2

a[2][0]=2

a[2][1]=2

a[2][2]=2

Not an Upper Triangular Matrix

SAMPLE OUTPUT 2:

Enter order of the matrix:3

Enter matrix elements:

a[0][0]=2

a[0][1]=2

a[0][2]=2

a[1][0]=0

a[1][1]=2

a[1][2]=2

a[2][0]=0

a[2][1]=0

a[2][2]=2

Upper Triangular Matrix

EXPERIMENT NO.4

AIM:

Program to find the factorial of a number using recursion

INTRODUCTION:

A factorial of a number n is the continued product n x (n-1) x (n-2)……..x 1

Recursively it could be written as

n!=n x (n-1)!

=n x (n-1) x (n-2)! …….

= n x (n-1) x (n-2) x……x 1!

Functions which call themselves repeatedly until a certain condition is met, are called recursive functions.

SOURCE CODE:

```
#include<iostream.h>
#include<conio.h>
int fact(int);
void main()
{     int n,ans;
      clrscr();
      cout<<"Enter any number:";
      cin>>n;
      ans=fact(n);                  //Calling recursive function fact(int)
      cout<<"\nFactorial of "<<n<<"="<<ans;
      getch();
}

int fact(int x)                     //Recursive function body
{     int y;
      if(x==0)
            return(1);
      y=fact(x-1);
      return(x*y);
}
```

SAMPLE OUTPUT:

Enter any number: 5

Factorial of 5=120

EXPERIMENT NO.5

AIM:

Program to create a stack and implement push and pop operations on it

INTRODUCTION:

A stack is a list of elements in which an element may be inserted or deleted only at one end, called the top of the stack. It works on the principle of Last In First Out. The insertion operation is termed as push and the deletion operation is termed as pop. In C/C++, array indexing begins at 0. So, initially Top pointer is kept at -1.

Algorithm for push operation:

1. **If Top=Maxsize-1**

   then print overflow and exit.

2. **Set Top=Top+1**
3. **Set stack[Top]=Item**
4. **Exit**

Algorithm for pop operation:

1. **If Top<0**

   then print underflow and exit.

2. **Item=stack[Top]**
3. **Set Top=Top-1**
4. **Return Item**
5. **Exit**

SOURCE CODE:

```
#include<iostream.h>
```

```
#include<conio.h>
#include<process.h>
#define MAX 10
void push(void);
int pop(void);
void traverse(void);
int stack[10];
int tos=-1;
void main()
{
      clrscr();
      char ch1='y';
      int ch;
      while(ch1=='y')
      {
            cout<<"\t\t\t\tSTACK";
            cout<<"\n 1.PUSH";
            cout<<"\n 2.POP";
            cout<<"\n 3.TRAVERSE";
            cout<<"\n Enter your choice:";
            cin>>ch;
            switch(ch)
            {
                  case 1:     push();
                              break;
                  case 2:     int p;
                              p=pop();
                              cout<<"\n The popped element is "<<p;
                              break;
                  case 3:     traverse();
                              break;
                  default:    cout<<"\n Wrong choice!!!!!!!!! ";
            }
            cout<<"\n Do you want to continue???? ";
            cin>>ch1;
      }
}
void push()                                   //Function to push element
{
      int item;
      if(tos==MAX)
      {
            cout<<"\n Stack full";
            getch();
            exit(0);
      }
      else
      {
            cout<<"\n Enter element to push: ";
            cin>>item;
            tos=tos+1;
            stack[tos]=item;
      }
}
int pop()                                     //Function to pop element
{
      int item;
```

```
        if(tos==-1)
        {
                cout<<"\n Stack empty";
                getch();
                exit(0);
        }
        else
        {
                item=stack[tos];
                tos=tos-1;
        }
        return(item);
}
void traverse()                         //Function to display stack elements
{
        if(tos==-1)
        {
                cout<<"\n Stack empty";
                getch();
                exit(0);
        }
        else
        {
                for(int i=tos; i>=0; i--)
                cout<<stack[i]<<"\t";
         }
}
```

SAMPLE OUTPUT:

STACK

1.PUSH

2.POP

3.TRAVERSE

Enter your choice:1

Enter element to push: 12

Do you want to continue???? y

STACK

1.PUSH

2.POP

3.TRAVERSE

Enter your choice:1

Enter element to push: 34

Do you want to continue???? y

STACK

1.PUSH

2.POP

3.TRAVERSE

Enter your choice:3

34 12

Do you want to continue???? y

STACK

1.PUSH

2.POP

3.TRAVERSE

Enter your choice:2

The popped element is 34

Do you want to continue???? y

STACK

1.PUSH

2.POP

3.TRAVERSE

Enter your choice:2

The popped element is 12

Do you want to continue???? y

STACK

1.PUSH

2.POP

3.TRAVERSE

Enter your choice:2

Stack empty

STACK Program in C

```
#include<stdio.h>
int stack[100],choice,n,top,x,i;
void push(void);
void pop(void);
void display(void);
int main()
{
   //clrscr();
   top=-1;
   printf("\n Enter the size of STACK[MAX=100]:");
   scanf("%d",&n);
   printf("\n\t STACK OPERATIONS USING ARRAY");
   printf("\n\t--------------------------------");
   printf("\n\t 1.PUSH\n\t 2.POP\n\t 3.DISPLAY\n\t 4.EXIT");
   do
   {
      printf("\n Enter the Choice:");
```

```
scanf("%d",&choice);
switch(choice)
{
   case 1:
   {
      push();
      break;
   }
   case 2:
   {
      pop();
      break;
   }
   case 3:
   {
      display();
      break;
   }
   case 4:
   {
      printf("\n\t EXIT POINT ");
      break;
   }
   default:
```

```
        {
            printf ("\n\t Please Enter a Valid Choice(1/2/3/4)");
        }

      }
   }
   while(choice!=4);
   return 0;
}
void push()
{
   if(top>=n-1)
   {
      printf("\n\tSTACK is over flow");

   }
   else
   {
      printf(" Enter a value to be pushed:");
      scanf("%d",&x);
      top++;
      stack[top]=x;
   }
}
```

```
void pop()
{
   if(top<=-1)
   {
      printf("\n\t Stack is under flow");
   }
   else
   {
      printf("\n\t The popped elements is %d",stack[top]);
      top--;
   }
}
void display()
{
   if(top>=0)
   {
      printf("\n The elements in STACK \n");
      for(i=top; i>=0; i--)
         printf("\n%d",stack[i]);
      printf("\n Press Next Choice");
   }
   else
   {
      printf("\n The STACK is empty");
```

EXPERIMENT NO.6

AIM:

Program to create a queue and implement insertion and deletion operations on it

INTRODUCTION:

A queue is a linear data structure that works on the principle of First In First Out. The element inserted first in the queue will be deleted first. Two variables front and rear would be implemented to keep a track of the inserted and deleted items. Initially both front and rear will be at -1. With every insertion rear will be incremented and with every deletion front will be incremented. Hence all insertions would take place at the rear end and all deletions will take place at the front end.

For n=10

Condition for empty queue:

front=rear=-1

Queue after 2 insertions:

2	3								

front=0 rear=1

Condition for queue full:

2	3	4	5	6	7	8	9	10	11

front=0 **rear=9**

Insertion Algorithm:

1. **If rear=max-1 then print "Queue full" and exit.**
2. **Set rear=rear+1**
3. **Set queue[rear]=item**
4. **Exit**

Deletion Algorithm:

1. **If front=rear then print "Queue empty" and exit**
2. **Set front=front+1**
3. **Return item=queue[front]**

4. **Exit**

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
#include<process.h>
int queue[5];
long front,rear;
void initqueue();
void display();
void main()
{
      int choice,info;
      clrscr();
      //Initialising queue
      initqueue();
      while(1)
      {
            //Displaying menu
            printf("\n             MENU             \n");
            printf("1.Insert an element in queue\n");
            printf("2.Delete an element from queue\n");
            printf("3.Display the queue\n");
            printf("4.Exit!\n");
            printf("Your choice: ");
            scanf("%i",&choice);
            switch(choice)
            {
                  case 1:     if(rear<4)
                              {
                                    printf("enter the number");
                                    scanf("%d",&info);

                                    if (front==-1)
                                    {
                                          front=0;
                                          rear=0;
                                    }

                                    else
                                          rear=rear+1;

                                    queue[rear]=info;
                              }
                              else
                              {
                                    printf("queue is full");
                                    getch();
                              }
                              break;


                  case 2: int info;
                              if(front!=-1)
                              {
```

```
                                info=queue[front];
                                if(front==rear)
                                {
                                      front=-1;
                                      rear=-1;
                                }
                                else
                                      front=front+1;

                                printf("no deleted is = %d",info);
                          }
                          else
                                printf("queue is empty");
                          getch();
                          break;
                    case 3:     display();
                                getch();
                                break;

                  case 4:       exit(0);
                                break;

                  default:printf("You entered wrong choice!");
                          getch();
                          break;
            }
      }
}
void initqueue()
{
//Initialising front & rear to -1
front=rear=-1;
}
/*displays the current position of the queue*/
void display()
{
      int i; //For loop driver
      //Displaying elements in queue
      for(i=front;i<=rear;i++)
            printf("%i\n",queue[i]);
}
```

SAMPLE OUTPUT:

MENU

1.Insert an element in queue

2.Delete an element from queue

3.Display the queue

4.Exit!

Your choice: 1

enter the number12

MENU

1.Insert an element in queue

2.Delete an element from queue

3.Display the queue

4.Exit!

Your choice: 1

enter the number45

MENU

1.Insert an element in queue

2.Delete an element from queue

3.Display the queue

4.Exit!

Your choice: 2

no deleted is = 12

MENU

1.Insert an element in queue

2.Delete an element from queue

3.Display the queue

4.Exit!

Your choice: 3

45

MENU

1.Insert an element in queue

2.Delete an element from queue

3.Display the queue

4.Exit!

Your choice: 4

C Program to Implement a Queue using an Array

```
#include <stdio.h>

 #define MAX 50

 void insert();

void delete();

void display();

int queue_array[MAX];

int rear = - 1;

int front = - 1;

main()

{

   int choice;

   while (1)

   {

      printf("1.Insert element to queue \n");

      printf("2.Delete element from queue \n");

      printf("3.Display all elements of queue \n");

      printf("4.Quit \n");

      printf("Enter your choice : ");
```

```
      scanf("%d", &choice);
      switch (choice)
      {
         case 1:
         insert();
         break;
         case 2:
         delete();
         break;
         case 3:
         display();
         break;
         case 4:
         exit(1);
         default:
         printf("Wrong choice \n");
      } /* End of switch */
   } /* End of while */
} /* End of main() */

void insert()
{
   int add_item;
   if (rear == MAX - 1)
```

```
    printf("Queue Overflow \n");
    else
    {
        if (front == - 1)
        /*If queue is initially empty */
        front = 0;
        printf("Inset the element in queue : ");
        scanf("%d", &add_item);
        rear = rear + 1;
        queue_array[rear] = add_item;
    }
} /* End of insert() */

void delete()
{
    if (front == - 1 || front > rear)
    {
        printf("Queue Underflow \n");
        return ;
    }
    else
    {
        printf("Element deleted from queue is : %d\n", queue_array[front]);
        front = front + 1;
```

```
    }
} /* End of delete() */


void display()
{
    int i;
    if (front == - 1)
        printf("Queue is empty \n");
    else
    {
        printf("Queue is : \n");
        for (i = front; i <= rear; i++)
            printf("%d ", queue_array[i]);
        printf("\n");
    }
}
```

Insertion Algorithm:

1. If rear=max-1 then print “Queue full” and exit.
2. Set rear=rear+1
3. Set queue[rear]=item
4. Exit

Deletion Algorithm:

1. If front=rear then print ”Queue empty” and exit
2. Set front=front+1
3. Return item=queue[front]
4. Exit

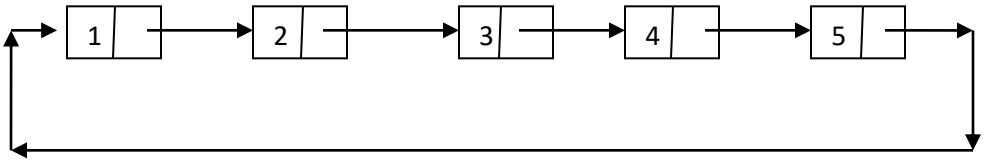
EXPERIMENT NO.7

AIM:

Program to create a circular queue and implement insertion and deletion operations on it

INTRODUCTION:

A circular queue is a queue in which the link of the last element points back to the first element.



SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
#define MAXSIZE 5
int cq[10];
int front=-1,rear=0;
void cqinsert();
int cqdelete();
void cqdisplay();
int choice;
char ch;
void main()
{
      clrscr();
      do
      {     printf("--------1. Insert---------\n");
            printf("------- 2. Delete---------\n");
            printf("------- 3. Display--------\n");
            printf("------- 4. Exit------------\n");
            printf("Enter your choice\n");
            scanf("%d",&choice);
            switch(choice)
            {
                  case 1 :    cqinsert();
                              break;
                  case 2 :    cqdelete();
                              break;
                  case 3 :    cqdisplay();
                              break;
                  case 4:     return;
            }
            fflush(stdin);
      }
      while(choice!=4);
}
```

```
void cqinsert()                //Function to insert element
{
      int num;
      if(front==(rear+1)%MAXSIZE)
      {
            printf("Queue is full\n");
            return;
      }
      else
      {
            printf("Enter the element to be inserted\n");
            scanf("%d",&num);
            if(front==-1)
                  front=rear=0;
            else
                  rear=(rear+1) % MAXSIZE;
                  cq[rear]= num;
      }
      return;
}
int cqdelete()                 //Function to delete element
{
      int num;
      if(front==-1)
      {
            printf("Queue is Empty\n");
            return 0;
      }
      else
      {
            num=cq[front];
            printf("Deleted element is =%d\n",cq[front]);
            if(front==rear)
                  front=rear=-1;
            else
                  front=(front+1)%MAXSIZE;
      }
      return(num);
}
void cqdisplay()               //Function to display queue elements
{
      int i;
      if(front==-1)
      {
            printf("Queue is empty\n");
            return;
      }
      else
      {
            printf("\nThe status of the queue\n");
                  for(i=front;i<=rear;i++)
                  {
                        printf("%d\n",cq[i]);
                  }
      }
      if(front>rear)
      {
```

```
            for(i=front;i<MAXSIZE;i++)
            {
                  printf("%d\n",cq[i]);
            }
            for(i=0;i<=rear;i++)
            {
                  printf("%d\n",cq[i]);
            }
      }
      printf("\n");
}
```

SAMPLE OUTPUT:

--------1. Insert---------

------- 2. Delete---------

------- 3. Display--------

------- 4. Exit----------

Enter your choice

1

Enter the element to be inserted

12

--------1. Insert---------

------- 2. Delete---------

------- 3. Display--------

------- 4. Exit----------

Enter your choice

1

Enter the element to be inserted

34

--------1. Insert---------

------- 2. Delete---------

------- 3. Display--------

------- 4. Exit----------

Enter your choice

2

Deleted element is =12

--------1. Insert---------

------- 2. Delete---------

------- 3. Display--------

------- 4. Exit----------

Enter your choice

1

Enter the element to be inserted

34

--------1. Insert---------

------- 2. Delete---------

------- 3. Display--------

------- 4. Exit----------

Enter your choice

3

The status of the queue

34

34

--------1. Insert---------

------- 2. Delete---------

------- 3. Display--------

------- 4. Exit----------

Enter your choice

EXPERIMENT NO.8

AIM:

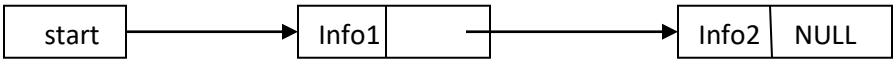
Program to create a linked list and implement insertion and deletion operations on it

INTRODUCTION:

Linked lists are list of data elements linked to one another. The logical ordering is represented by having each element pointing to the next element. Each element is called a node, which has two parts:

i) info part which stores the information

ii) next part which points to the next element



SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
#include<malloc.h>
struct node
{
int info;
struct node *next;
};
typedef struct node NODE;
NODE *start;
void traversinorder(NODE *start)
{
      while(start != (NODE *) NULL)
      {
            printf("%d\n",start->info);
            start=start->next;
      }
}
void insertatbegin(int item)
{
      NODE *ptr;
      ptr=(NODE *)malloc(sizeof(NODE));
      ptr->info=item;
      if(start==(NODE *)NULL)
      ptr->next=(NODE *)NULL;
      else
      ptr->next=start;
```

```
        start=ptr;
}
void insert_at_end(int item)
{
        NODE *ptr,*loc;
        ptr=(NODE *)malloc(sizeof(NODE));
        ptr->info=item;
        ptr->next=(NODE *)NULL;
        if(start==(NODE*)NULL)
        start=ptr;
        else
        {
                loc=start;
                while(loc->next!=(NODE *)NULL)
                loc=loc->next;
                loc->next=ptr;
        }
}

void dele_beg(void)
{
        NODE *ptr;
        if(start==(NODE *)NULL)
                return;
        else
        {
                ptr=start;
                start=(start)->next;
                free(ptr);
        }
}
void dele_end(NODE *start)
{
        NODE *ptr,*loc;
        if(start==(NODE *)NULL)
                return;
        else if((start)->next==(NODE *)NULL)
        {
                ptr=start;
                start=(NODE *)NULL;
                free(ptr);
        }
        else
        {
                loc=start;
                ptr=(start)->next;
                while(ptr->next!=(NODE *)NULL)
                {
                        loc=ptr;
                        ptr=ptr->next;
                }
                loc->next=(NODE *)NULL;
                free(ptr);
        }
}
void main()
{
```

```
        int choice,item,after;
        char ch;
        clrscr();
        start=NULL;
        do
        {     printf("\t\t\t\tLINKED LIST\n");
              printf("1.Insert element at begining \n");
              printf("2.Insert element at end \n");
              printf("3.Traverse the linked list\n");
              printf("4.Delete from the begining\n");
              printf("5.Delete from the last\n");
              printf("6.Exit\n");
              printf("Enter your choice:");
              scanf("%d",&choice);
              switch(choice)
              {
                    case 1: printf("Enter the item:");
                          scanf("%d",&item);
                          insertatbegin(item);
                          break;
                    case 2: printf("Enter the item:");
                          scanf("%d",&item);
                          insert_at_end(item);
                          break;
                    case 3: printf("\nTraversing the linked list:\n");
                          traversinorder(start);
                          break;
                    case 4: printf("Deleted the item\n");
                          dele_beg();
                          break;
                    case 5: printf("Deleted the item\n");
                          dele_end(start);
                          break;
                    case 6: return;
              }
              fflush(stdin);
              printf("Do your want to continue(y/n)?\n");
              scanf("%c",&ch);
        }while((ch=='y')||(ch=='y'));

}
```

SAMPLE OUTPUT:

LINKED LIST

1.Insert element at begining

2.Insert element at end

3.Traverse the linked list

4.Delete from the begining

5.Delete from the last

6.Exit

Enter your choice:1

Enter the item:34

Do your want to continue(y/n)?

y

LINKED LIST

1.Insert element at begining

2.Insert element at end

3.Traverse the linked list

4.Delete from the begining

5.Delete from the last

6.Exit

Enter your choice:2

Enter the item:56

Do your want to continue(y/n)?

y

LINKED LIST

1. Insert element at begining

2.Insert element at end

3.Traverse the linked list

4.Delete from the begining

5.Delete from the last

6.Exit

Enter your choice:3

Traversing the linked list:

34

56

Do your want to continue(y/n)?

y

LINKED LIST

1.Insert element at begining

2.Insert element at end

3.Traverse the linked list

4.Delete from the begining

5.Delete from the last

6.Exit

Enter your choice:4

Deleted the item

Do your want to continue(y/n)?

n

EXPERIMENT NO.9

AIM:

Program to traverse a binary tree in pre-order, in-order and post-order

INTRODUCTION:

Inorder traversal (Symmetric order)

1) **Traverse (inorder) the left sub tree**
2) **Visit the root node**
3) **Traverse (inorder) the right sub tree**

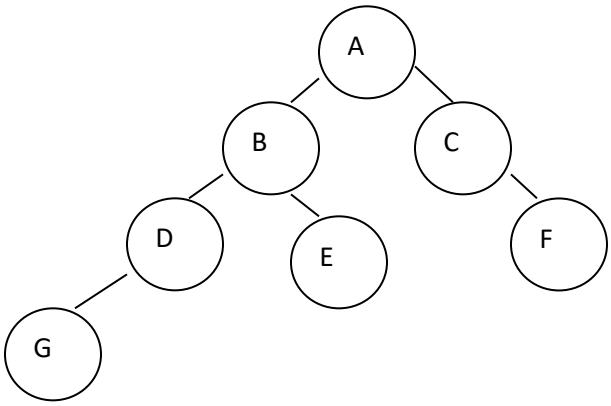
Preorder traversal

1) **Visit the root node**
2) **Traverse(preorder) the left sub tree**
3) **Traverse (preorder) the right sub tree**

Postorder traversal

1) **Traverse(postorder) the left sub tree**
2) **Traverse (postorder) the right sub tree**
3) **Visit the root node**

Example:



Inorder: GDBEACF

Preorder: ABDGECF

Postorder: GDEBFCA

SOURCE CODE :

```
#include<iostream.h>
#include<stdio.h>
#include<process.h>
#include<conio.h>
#include<alloc.h>
struct rec
{
      long num;
      struct rec *left;
      struct rec *right;
};
struct rec *tree=NULL;
struct rec *insert(struct rec *tree,long num);
int select();
void preorder(struct rec *tree);
void inorder(struct rec *tree);
void postorder(struct rec *tree);
int count=1;
void main()
{
      clrscr();
      int choice;
      long digit;
      do
       {
         choice=select();
         switch(choice)
            {
            case 1: puts("Enter integer: To quit enter 0");
                  cin>>digit;
                  while(digit!=0)
                      {
                        tree=insert(tree,digit);
                        cin>>digit;
                      }continue;
            case 2: puts("\npreorder traversing TREE");
                  preorder(tree);continue;
            case 3: puts("\ninorder traversing TREEE");
                  inorder(tree);continue;
            case 4: puts("\npostorder traversing TREE");
                  postorder(tree);continue;
            case 5: puts("END");
            exit(0);
            }
      }while(choice!=5);
}
int select()
{
int selection;
      do
        {
            puts("\nEnter 1: Insert a node in the BT");
            puts("Enter 2: Display(preorder)the BT");
            puts("Enter 3: Display(inorder)the BT");
```

```
            puts("Enter 4: Display(postorder)the BT");
            puts("Enter 5: END");
            puts("Enter your choice");
            cin>>selection;
                 if((selection<1)||(selection>5))
                  {
                    puts("wrong choice:Try again");
                    getch(); }
         }while((selection<1)||(selection>5));
         return (selection);
}
struct rec *insert(struct rec *tree,long digit)
{
      if(tree==NULL)
        {
            tree=(struct rec *)malloc(sizeof(struct rec));
            tree->left=tree->right=NULL;
            tree->num=digit;count++;
        }
else
      if(count%2==0)
      tree->left=insert(tree->left,digit);
else
      tree->right=insert(tree->right,digit);
      return(tree);
}
void preorder(struct rec *tree)
{
      if(tree!=NULL)
        {
            cout<<"\n"<<tree->num;
            preorder(tree->left);
            preorder(tree->right);
        }
}
void inorder(struct rec *tree)
{
      if(tree!=NULL)
          {
            inorder(tree->left);
            cout<<"\n"<<tree->num;
            inorder(tree->right);
          }
}
void postorder(struct rec *tree)
{
      if(tree!=NULL)
        {
            postorder(tree->left);
            postorder(tree->right);
            cout<<"\n"<<tree->num;
        }

}
```

SAMPLE OUTPUT:

Enter 1: Insert a node in the BT

Enter 2: Display(preorder)the BT

Enter 3: Display(inorder)the BT

Enter 4: Display(postorder)the BT

Enter 5: END

Enter your choice

1

Enter integer: To quit enter 0

23

45

67

0

Enter 1: Insert a node in the BT

Enter 2: Display(preorder)the BT

Enter 3: Display(inorder)the BT

Enter 4: Display(postorder)the BT

Enter 5: END

Enter your choice

2

preorder traversing TREE

23

45

67

Enter 1: Insert a node in the BT

Enter 2: Display(preorder)the BT

Enter 3: Display(inorder)the BT

Enter 4: Display(postorder)the BT

Enter 5: END

Enter your choice

3

inorder traversing TREEE

45 23 67

Enter 1: Insert a node in the BT

Enter 2: Display(preorder)the BT

Enter 3: Display(inorder)the BT

Enter 4: Display(postorder)the BT

Enter 5: END

Enter your choice 5

END

EXPERIMENT NO.10

AIM:

Program to implement sorting of data using:

a) Bubble sort

b) Selection sort

c) Insertion sort

INTRODUCTION:

Bubble Sort

Multiple swapping take place in one pass. Smaller elements move or bubble up to the top of the list. Adjacent members of the list to be sorted are compared. For obtaining ascending order, if the item on left is greater than the item immediately right to it, they are swapped. This process is carried on till the list is sorted.

Example:

List:	**85**	**66**	**53**	**33**	**27**
Pass I	**66**	**53**	**33**	**27**	**85**
Pass II	**53**	**33**	**27**	**66**	**85**
Pass III	**33**	**27**	**53**	**66**	**85**
Pass IV	**27**	**33**	**53**	**66**	**85**

Selection Sort

Perform a search through the table starting from the first record to locate the element with the smallest key. Interchange it with the first record. Thus, the smallest key is placed in the first position. In the second iteration, locate the second smallest key, examining the keys of the records starting from the second record onwards. Interchange it with the second record. Continue the process until all records are sorted.

Example:

List:	**45**	**25**	**75**	**15**	**65**	**55**	**95**	**35**
Pass I	**15**	**25**	**75**	**45**	**65**	**55**	**95**	**35**

Pass II	**15**	**25**	**75**	**45**	**65**	**55**	**95**	**35**
Pass III	**15**	**25**	**35**	**45**	**65**	**55**	**95**	**75**
Pass IV	**15**	**25**	**35**	**45**	**65**	**55**	**95**	**75**
Pass V	**15**	**25**	**35**	**45**	**55**	**65**	**95**	**75**
Pass VI	**15**	**25**	**35**	**45**	**55**	**65**	**95**	**75**
Pass VII	**15**	**25**	**35**	**45**	**55**	**65**	**75**	**95**

Insertion Sort:

Suppose an array A with n elements A[1], A[2].....A[n] is in memory. The insertion sort algorithm scans A from A[1] to A[n], inserting each element A[k] into its proper position in the previously sorted sub array A[1], A[2].... A[k-1].

Example:

List:	**77**	**33**	**44**	**11**	**88**	**22**	**55**
Pass I	**77**	**33**	**44**	**11**	**88**	**22**	**55**
Pass II	**33**	**77**	**44**	**11**	**88**	**22**	**55**
Pass III	**33**	**44**	**77**	**11**	**88**	**22**	**55**
Pass IV	**11**	**33**	**44**	**77**	**88**	**22**	**55**
Pass V	**11**	**33**	**44**	**77**	**88**	**22**	**55**
Pass VI	**11**	**22**	**33**	**44**	**77**	**88**	**55**
Pass V	**11**	**22**	**33**	**44**	**55**	**77**	**88**

SOURCE CODE:

Experiment 11 (a): Program to implement Bubble Sort

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[100],n,i,j,temp;
clrscr();
```

```
printf("How many elements:");
scanf("%d",&n);
printf("Enter the element of array:\n");
for(i=0;i<=n-1;i++)
 {
  scanf("%d",&a[i]);
 }
for(i=0;i<=n-1;i++)
{
  for(j=0;j<=n-1-i;j++)
     {
      if(a[j]>a[j+1])
      {
       temp=a[j];
       a[j]=a[j+1];
       a[j+1]=temp;
      }
      }
}
printf("Elements of array after bubble sorting are:\n");
for(i=0;i<=n-1;i++)
{
printf("%d\n",a[i]);
}
getch();
}
```

SAMPLE OUTPUT:

How many elements:5

Enter the element of array:

12

34

23

56

45

Elements of array after bubble sorting are:

12

23

34

45

56

Experiment 11 (b): Program to implement Selection Sort

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[100],n,i,j,temp,loc,min;
clrscr();
printf("\nEnter number of elements:\n");
scanf("%d",&n);
printf("Enter the elements of array:\n");
  for(i=0;i<=n-1;i++)
  {
     scanf("%d",&a[i]);
     }
     min=a[0];
for(i=0;i<=n-1;i++)
   {
    min=a[i];
    loc=i;
    for(j=i+1;j<=n-1;j++)
       {
      if(a[j]<min)
       {
        min=a[j];
        loc=j;
       }
       }
if(loc!=1)
   {
     temp=a[i];
     a[i]=a[loc];
     a[loc]=temp;
   }

}
printf("The array after selection sort is:\n");
for(i=0;i<=n-1;i++)
{
printf("%d\n",a[i]);
}
getch();
}
```

SAMPLE OUTPUT:

Enter number of elements:

7

Enter the elements of array:

12

34

2

3

6

7

8

The array after selection sort is:

2

3

6

7

8

12

34

Experiment 11 (c): Program to implement Insertion Sort

```
#include<stdio.h>
#include<conio.h>
void main()
{
int a[100],n,k,i,j,temp;
clrscr();
printf("How many elements:\n");
scanf("%d",&n);
printf("Enter the elements of array:");
for(i=0;i<=n-1;i++)
{
scanf("%d",&a[i]);
}
for(k=1;k<=n-1;k++)
{
temp=a[k];
j=k-1;
while((temp<a[j])&&(j>=0))
{
```

```
a[j+1]=a[j];
j=j-1;
}
a[j+1]=temp;
}
printf("Elements of array after insertion sort:\n");
for(i=0;i<=n-1;i++)
{
printf("%d\n",a[i]);
}
getch();
}
```

SAMPLE OUTPUT:

How many elements:

5

Enter the elements of array:78

45

34

12

67

Elements of array after insertion sort:

12 34 45 67 78

EXPERIMENT NO.11

AIM:

Program to implement a graph and traverse it using Breadth First Search

INTRODUCTION:

A graph is a structure G={V,E} in which V is a finite set of nodes and E is a finite set of edges. It is represented by an adjacency matrix. An adjacency matrix for a graph with n nodes is an nxn matrix. Any element of the adjacency matrix is either 0 or 1. A_{ij} =1 if there is an edge from V_i to V_j and A_{ij}=0 if there is no such edge.

Breadth First Search:

Step 1: Start with any vertex and mark it as visited.

Step 2: Using the adjacency matrix of the graph, find a vertex adjacent to the vertex in step 1. Mark it as visited.

Step 3: Return to vertex in step 1 and move along an edge towards an unvisited vertex, and mark the new vertex as visited.

Step 4: Repeat step 3 until all vertices adjacent to the vertex, as selected in step 2, have been marked as visited.

Step 5: Repeat step 1 through step 4 starting from the vertex visited in step 2, then starting from the nodes visited in step 3 in the order visited. If all vertices have been visited, then continue to next step.

Step 6: Stop

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
void insert(int);
int q[20],r=-1,g[7][7],row;
void insert(int x)
{
      r++;
      q[r]=x;
}
remove()
{
      int item,k;
```

```
        item=q[0];
        for(k=0;k<r;k++)
                q[k]=q[k+1];
        r--;
        return(item);
}
void main()
{
        int i,j, num,w,visited[10],v,j1;
        int l,vertices[10],count=0,final[10];
        clrscr();
        randomize();
        printf("Enter no. of vertices:");
        scanf("%d", &row);
        printf("\nAdjacency Matrix:\n\n");
        printf("    ");
        for(j=0;j<row;j++)
                printf("  Vertex %d ",j);
        for(i=0;i<row;i++)
        {
                for(j=count;j<row;j++)
                {
                        if(i!=j)
                        {
                                g[i][j]=random(2);
                                g[j][i]=g[i][j];
                        }
                        else
                                g[i][j]=0;
                }
                count++;
        }
        for(i=0;i<row;i++)
        {
                printf("\nVertex%d",i);
                for(j=0;j<row;j++)
                        printf("%8d",g[i][j]);
                printf("\n\n");
        }
        for(i=0;i<row;i++)
                visited[i]=0;
        printf("\n Enter start vertex:");
        scanf("%d",&v);
        visited[v]=1;insert(v);
        getch();
        clrscr();
        printf("\nStart vertex=V%d\n\n",v);
        count=1;
        j1=0;
        while(r>=0)
        {
                v=remove();
                final[j1]=v;
                j1++;
                l=0;
                for(i=0;i<row;i++)
                        if(g[v][i]==1)
```

```
                {
                        vertices[l]=i;
                        l++;
                }
            for(i=0;i<l;i++)
            {
                w=vertices[i];
                printf("Step %d:Vertex visited: Vertex %d\n",count,w);
                if(visited[w]!=1)
                {
                        insert(w);
                        visited[w]=1;
                }
            }
            printf("Elements in the queue:");
            if(r>=0)
            for(j=1;j<=r;j++)
                printf("%d",q[j]);
            else
                printf("Traversal complete");
            count++;
            printf("\n\n");
            getch();
            clrscr();
        }
        printf("BFS Traversal:\n");
        if(count==2)
            printf("\nIsolated vertex");
        else
            for(i=0;i<j1;i++)
                printf("Vertex %d ",final[i]);
        getch();
}
```

SAMPLE OUTPUT:

Enter no. of vertices:4

Adjacency Matrix:

	Vertex 0	Vertex 1	Vertex 2	Vertex 3
Vertex0	0	1	1	1
Vertex1	1	0	0	1
Vertex2	1	0	0	0
Vertex3	1	1	0	0

Enter start vertex:2

Start vertex=V2

Step 1:Vertex visited: Vertex 0

Elements in the queue:

Step 2:Vertex visited: Vertex 1

Step 2:Vertex visited: Vertex 2

Step 2:Vertex visited: Vertex 3

Elements in the queue:3

Step 3:Vertex visited: Vertex 0

Step 3:Vertex visited: Vertex 3

Elements in the queue:

Step 4:Vertex visited: Vertex 0

Step 4:Vertex visited: Vertex 1

Elements in the queue: Traversal complete

BFS Traversal:

Vertex 2 Vertex 0 Vertex 1 Vertex 3